MIZAN - TEPI UNIVERSITY

TEPI CAMPUS

SCHOOL OF COMPUTING AND INFORMATICS

DEPARTMENT OF INFORMATION SYSTEMS

INDIVIDUAL ASSIGNMENT OF Object Oriented Programing

COURSE CODE : INSY2042

Section A



Name	Id
Gebrie Adane	MTUUR/7832/17

Submited To:Mr.samson

Submition Date:25/7/2018 E.C

Content **page**

Introduction…….I

1.File Handling in Java……..1

1.1 Create a File……..1

1.2 Deleting a File……..2

1.3 Manipulation ……..3

Types of File Manipulation in Java……3

2.Remote Method Invocation in Java……7

2.1 Working of RMI……7

2.2 Overview of RMI Architecture……...10

2.3 RMI Architecture diagram……10

Conclusion……..14

References……15

Introduction

Remote Method Invocation (RMI) is a Java API that enables communication between objects located in different Java Virtual Machines (JVMs), often across separate physical machines. It was designed to simplify the development of distributed applications by allowing programmers to invoke methods on remote objects as if they were local. This abstraction hides the complexity of networking, serialization, and communication protocols, making distributed computing more accessible.

The RMI architecture is built around several key components:

Client Application: Initiates the request to access a remote service.

- **Client Application**: Initiates the request to access a remote service.
- **Stub**: Acts as a proxy on the client side, handling method calls and forwarding them to the server.
- **Skeleton**: Receives requests on the server side, unpacks the data, and invokes the actual method on the remote object.
- **Remote Object**: The actual object whose methods are executed remotely.
- **RMI Registry**: A naming service that allows clients to locate remote objects.
- **Communication Layer**: Manages the transport of data between client and server.

1 Explain File management in Java. Describe how to create, delete, and manipulate files using Java.

1 .File Handling in Java In Java, file handling means working with files like creating them, reading data, writing data or deleting them. It helps a program save and use information permanently on the computer.

File Operations

1.1 Create a File

In order to create a file in Java, you can use the createNewFile() method. If the file is successfully created, it will return a Boolean value true and false if the file already exists.

example

```
import java.io.File;
import java.io.IOException;
public class CreateFile
{
public static void main(String[] args)
   {
  try {
      }
   }
}
```

```
        File Obj = new File("myfile.txt");

        // Creating File
        if (Obj.createNewFile()) {
            System.out.println("File created: " + Obj.getName());
        }
        else {
            System.out.println("File already exists.");
        }
      }
    // Exception Thrown
      catch (IOException e) {
  System.out.println("An error has occurred.");
        e.printStackTrace();
```

Output:

```
C:\Exception_Handling>javac WriteFile.java

C:\Exception_Handling>java WriteFile
Successfully written.
```

1.2 Deleting a File

Deleting a file means removing an existing file permanently from the system.
In Java, file deletion is done using the delete() method of the File class. If the file exists and is deleted → returns true
If the file does not exist or deletion fails → returns false
This operation is used to remove unnecessary or unwanted files.
Deleted files cannot be easily recovered
Always check if the file exists before deleting
We use the delete() method in order to delete a file.
Example

```
 import java.io.File;

public class DeleteFile
{
   public static void main(String[] args)
{
      File Obj = new File("myfile.txt");

      // Deleting File
      if (Obj.delete()) {
         System.out.println("The deleted file is : " + Obj.getName());
      }
      else {
         System.out.println(
            "Failed in deleting the file.");
```

```
    }
  }
}
```

output

```
C:\Exception_Handling>javac DeleteFile.java

C:\Exception_Handling>java DeleteFile
The deleted file is : myfile.txt
```

1.3 Manipulation

General Definition: Manipulation is the process of handling, controlling, or modifying something to achieve a desired outcome.
In File Management (Java): Manipulation refers to the actions performed on files such as reading data, writing data, renaming, copying, moving, or changing file attributes.

1.4 Types of File Manipulation in Java

1.Reading *definition*

Reading in file management is the process of extracting data from a file, which means that the program opens the file and retrieves the information stored inside so that it can be displayed to the user, processed by the application, or used in further calculations and decision-making

Importance of reading

The importance of reading lies I n the fact that without the ability to retrieve data from files, programs would not be able to access previously stored information, making it impossible to analyze logs, load saved documents, or continue work from where a user left off in earlier sessions.

Example

```
import java.io.FileReader;
import java.io.IOException;
```

```
public class ReadExample {
   public static void main(String[] args) {
      try {
         FileReader reader = new FileReader("example.txt");
         int ch;
         while ((ch = reader.read()) != -1) {
            System.out.print((char) ch);
}
         reader.close();
      } catch (IOException e) {
         e.printStackTrace();
      }
   }
}
```

2.Writing

Writing in file management refers to the process of adding new content or updating existing content in a file, which allows data to be permanently stored on disk so that it can be accessed later even after the program has been closed or the computer has been restarted

- **Importance of writing**

Writing is important because it enables applications to save user input, generate reports, maintain logs, and preserve information for future use, making it a fundamental operation for any software that requires persistent data storage and long-term record keeping.

Example

```
import java.io.FileWriter;
import java.io.IOException;

public class WriteExample {
public static void main(String[] args) {
try {
FileWriter writer = new FileWriter("example.txt");
writer.write("Hello, this is written into the file!");
writer.close();
System.out.println("Successfully wrote to the file.");
```

```
} catch (IOException e) {
e.printStackTrace();
}
}
}
```

3.Renaming

Renaming in file management is the process of changing the name of a file while keeping its content and location intact, which means that the file remains the same internally but is identified diffImportance
Renaming is important because it helps organize files more effectively, allows users to apply meaningful names for easier identification, and supports version control by distinguishing between different iterations of the same file without altering the actual content inside.
erently in the file system for easier recognition and organization.

Example

```
import java.io.File;

public class RenameExample {
   public static void main(String[] args) {
      File oldFile = new File("example.txt");
      File newFile = new File("renamed.txt");
      if (oldFile.renameTo(newFile)) {
         System.out.println("File renamed successfully.");
      } else {
         System.out.println("Failed to rename the file.");
      }
   }
}
```

4.Moving

Moving in file management refers to relocating a file from one directory or location to another, which removes the file from its original place and places it in

the new destination, thereby reorganizing the file structure without altering the file's content.

- **Importance of moving**

Copying and moving are important because they allow users to back up critical files, share data across different folders or systems, and reorganize file structures to maintain a clean and logical arrangement of information that supports efficiency and reliability.

Example

```
import java.io.IOException;

import java.nio.file.*;


public class CopyMoveExample {
   public static void main(String[] args) {
      Path source = Paths.get("example.txt");
      Path copyDestination = Paths.get("copy_example.txt");
      Path moveDestination = Paths.get("moved_example.txt");

      try {
         // Copying the file
         Files.copy(source, copyDestination,
StandardCopyOption.REPLACE_EXISTING);
         System.out.println("File copied successfully.");

         // Moving the file
         Files.move(source, moveDestination,
StandardCopyOption.REPLACE_EXISTING);
         System.out.println("File moved successfully.");
      } catch (IOException e) {
         e.printStackTrace();
      }
   }
```

}

2.What is Remote Method Invocation (RMI)? Explain its architecture and advantages.

2.Remote Method Invocation in Java

Remote Method Invocation (RMI) is an API that allows an object to invoke a method on an object that exists in another address space, which could be on the same machine or on a remote machine. Through RMI, an object running in a JVM present on a computer (Client-side) can invoke methods on an object present in another JVM (Server-side). RMI creates a public remote server object that enables client and server-side communications through simple method calls on the server object.

Stub Object: The stub object on the client machine builds an information block and sends this information to the server.

The block consists of

An identifier of the remote object to be used

Method name which is to be invoked

Parameters to the remote JVM

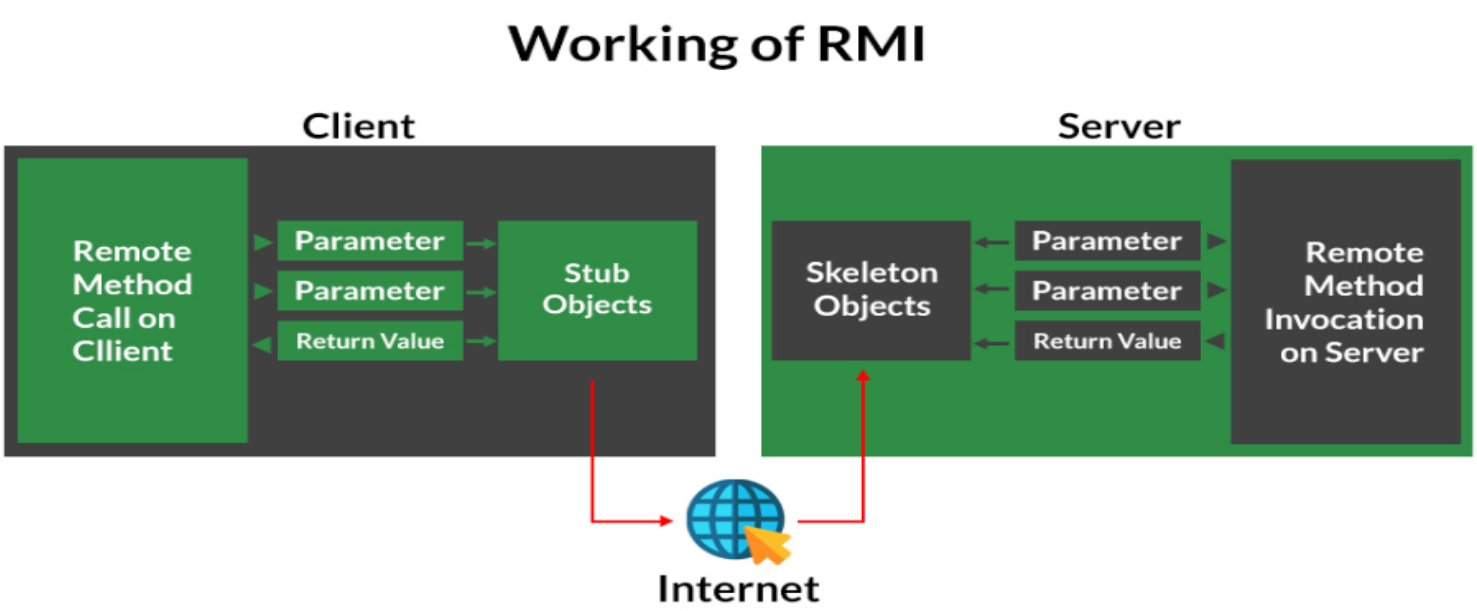
Skeleton Object: The skeleton object passes the request from the stub object to the remote object. It performs the following tasks

It calls the desired method on the real object present on the server.

It forwards the parameters received from the stub object to the method.

2.1 Working of RMI

The communication between client and server is handled by using two intermediate objects: Stub object (on client side) and Skeleton object (on server-side) as also can be depicted from below media as follows:



These are the steps to be followed sequentially to implement Interface as defined below as follows:
Defining a remote interface
Implementing the remote interface
Creating Stub and Skeleton objects from the implementation class using rmic (RMI compiler)
Start the rmiregistry
Create and execute the server application program
Create and execute the client application program.

Step 1: Defining the remote interface
The first thing to do is to create an interface that will provide the description of the methods that can be invoked by remote clients. This interface should extend the Remote interface and the method prototype within the interface should throw the RemoteException.
example

```
import java.rmi.Remote;
import java.rmi.RemoteException;

public interface Hello extends Remote {
   String sayHello() throws RemoteException;
}
```

Step 2: Implementing the remote interface
The next step is to implement the remote interface. To implement the remote

interface, the class should extend to UnicastRemoteObject class of java.rmi package. Also, a default constructor needs to be created to throw the java.rmi.RemoteException from its parent constructor in class.

example

```
import java.rmi.server.UnicastRemoteObject;
import java.rmi.RemoteException;

public class HelloImpl extends UnicastRemoteObject implements Hello {
   protected HelloImpl() throws RemoteException {
      super();
   }

   public String sayHello() throws RemoteException {
      return "Hello, world!";
   }
}
```

Step 3: Creating Stub and Skeleton objects from the implementation class using rmic

The rmic tool is used to invoke the rmi compiler that creates the Stub and Skeleton objects. Its prototype is rmic classname. For above program the following command need to be executed at the command prompt

rmic SearchQuery.

Step 4: Start the rmiregistry

Start the registry service by issuing the following command at the command prompt start rmiregistry

Step 5: Create and execute the server application program

The next step is to create the server application program and execute it on a separate command prompt.

The server program uses createRegistry method of LocateRegistry class to create rmiregistry within the server JVM with the port number passed as an argument. The rebind method of Naming class is used to bind the remote object to the new name.

Step 6: Create and execute the client application program

The last step is to create the client application program and execute it on a

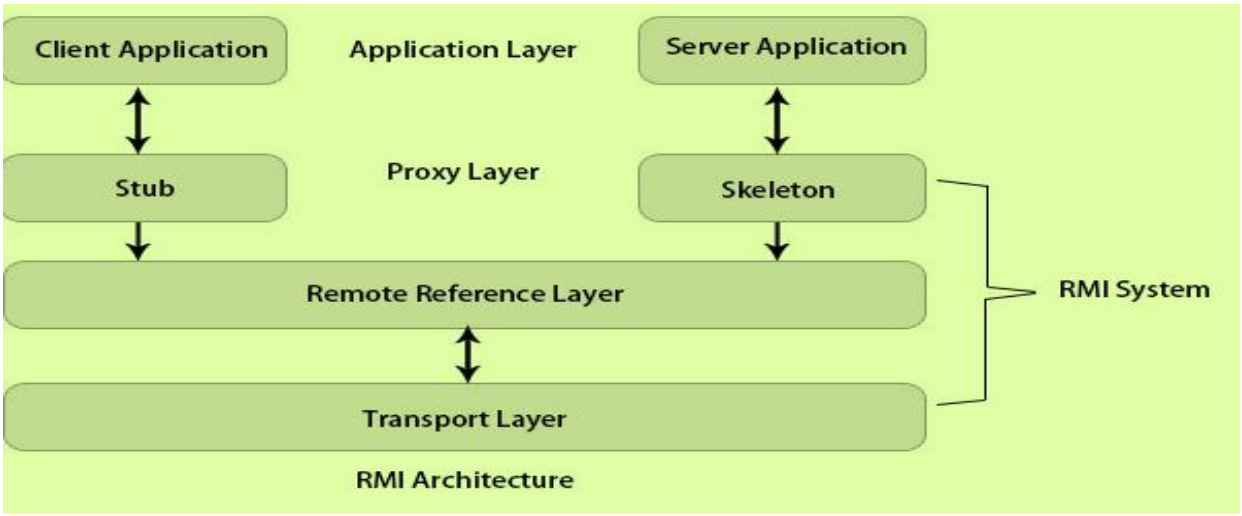
separate command prompt . The lookup method of the Naming class is used to get the reference of the Stub object.

2.2 Overview of RMI Architecture

application architecture, it is always a need for communication between two different applications. In Java-based applications, one application communicates with another remote/different application running distributed somewhere else using an RMI architecture mechanism.
RMI stands for Remote Method Invocation. Java provides an API allowing an object residing in one JVM (Java Virtual Machine) to access or invoke an object running on another. The other JVM could be on the same machine or a remote machine. This is an interesting feature because, in real-time applications, it becomes very easy for Java applications to communicate directly with each other without any external communication mechanism. Also, it is always a need for secure communication between applications based on distributed application architecture.
The client and server applications are the respective JVMs of the client machine and server machines. In the RMI application, we write two programs: the client program, which resides on the client, and the server program, which resides on the server machine.

2.3 RMI Architecture diagram

Below is a diagram of RMI architecture in a simple way. You will find various forms of the same architecture on the internet, but we have a simple one to help explain it better.



1. Application Layer
This layer is the actual systems, i.e. client and server, which are involved in communication. The Java program on the client side communicates with the Java program on the server side.
2. Stub
We have client objects from the design intro; In RMI architecture, it is known as Stub. It is an object that resides on the client machine and acts as a proxy for the remote object. It is like a gateway for the client program.
The stub has the same methods as a remote object. When the client calls on the stub object, the stub forwards this request to a remote object (Skeleton) via RMI infrastructure, which is then executed on the server.
Stub Performs the following events:
Initiates connection with remote JVM.
Writes and transmits (Marshals) parameters to remote JVM.

Waits for the result.

Reads (Unmarshalls) the returned result.

- Pass the received result to the caller.

3. **Skeleton**

The server object, which is located in a server machine, is referred to as the Skeleton. Stub communicates with the server application with the help of an intermediate Skeleton object.
The responsibility of the skeleton object is to send parameters to method implementation and send the return values back to the client.
Skeleton Performs the following events:

4**.Stub / Skeleton layer**

The Proxy Layer, also known as the Stub/Skeleton layer, intercepts calls made by the client and redirects them to the remote object. Stub and Skeleton are the proxies for the client and server. The Stub and Skeleton objects are like an interface between an application and the rest of the RMI System.

5. **Remote Reference Layer**

The Remote Reference Layer connects the proxy layer to the RMI mechanism. This layer is responsible for communicating and transferring objects between client and server. This layer defines and supports the invocation semantics of the RMI connection.
The remote Reference Layer maintains the session during the method call. i.e., It manages the references made by the client to the remote server object. This layer is also responsible for handling duplicated objects.

6**. Transport Layer**

The transport layer is responsible for setting up communication between the two machines. This layer uses standard **TCP/IP protocol** for connection. This layer performs the actual transportation of data. This layer is part of the Remote Reference Layer.

Advantages of RMI

1. **Simplicity in Distributed Programming**

RMI makes distributed computing feel natural because you call methods on remote objects just like local ones.
Developers don't need to manually handle sockets, streams, or low-level networking code.

2. **Object-Oriented Communication**

Unlike traditional message-passing systems, RMI allows entire Java objects (not just primitive data types) to be passed between JVMs.
This supports **object serialization**, meaning complex data structures can be transmitted seamlessly.

3. **Platform Independence**

Since RMI is built entirely in Java, applications are portable across different operating systems.
It aligns with Java's philosophy: *"Write Once, Run Anywhere."*

4. **Transparency**

The complexity of remote communication (marshalling, unmarshalling, transport protocols) is hidden from the developer.
From the programmer's perspective, invoking a remote method looks identical to invoking a local one.

5. **Security**

RMI integrates with Java's security manager and policy files.
It ensures safe downloading of classes and prevents unauthorized access to system resources.

6. **Reusability of Existing Code**

Existing Java classes can be easily adapted into distributed applications by making them remote objects.
This reduces development effort and promotes modular design.

7. **Scalability**

Multiple clients can access remote objects simultaneously.
RMI supports building scalable distributed systems where services can be spread across multiple servers.

8. **Flexibility and Extensibility**

RMI can be combined with other Java technologies (like JDBC for databases or Servlets for web applications).
It allows developers to extend applications into distributed environments without rewriting core logic.

9. **Integration with Naming Service** The RMI Registry provides a simple way to locate remote objects.This makes service discovery straightforward compared to manual IP/port management.

10. **Automatic Garbage Collection**

RMI supports distributed garbage collection, meaning unused remote objects can be cleaned up automatically.
This helps manage memory efficiently across JVMs.

❖ Conclusion

In conclusion, RMI architecture provides a structured and efficient way to build distributed applications in Java. By dividing responsibilities among the client, stub, skeleton, remote object, and registry, RMI ensures that developers can focus on application logic rather than the complexities of network communication. The stub and skeleton act as intermediaries, handling the technical details of marshaling and unmarshaling data, while the registry simplifies object discovery.

This layered design not only promotes modularity and scalability but also enhances reliability in distributed systems. With RMI, developers can achieve transparent communication between different JVMs, making it a cornerstone technology for building robust, enterprise-level applications.

applications. Understanding the architecture in detail helps us appreciate how RMI bridges the gap between local and remote execution, enabling Java to support true distributed computing.

References

1. File Handling: https://docs.oracle.com/javase/tutorial/essential/io/RMI
2. File Handling in Java: https://www.geeksforgeeks.org/file-handling-in-java/
3. RMI in Java: https://www.geeksforgeeks.org/java-rmi/